

Availability and Reproducibility Initiative Report

1. Source Code

- [Source code](#)

2. Software/Library Dependencies

- Clang++
- CMake (version 3.22 or higher)
- Boost (required for end-to-end benchmarks)
- BZIP2 (required for PED benchmarks)
- OpenSSL (required for PED benchmarks)
- libcurl (required for PED benchmarks)
- g++ (required for PED benchmarks)
- pandas (required for plots)
- seaborn (required for plots)
- Java 8 (required for ELF)
- [Maven 3.9.9](#) (required for ELF)

This [script](#) can be used to install all necessary dependencies on Ubuntu.

3. Hardware Requirements

- [Intel Ice Lake](#) (x86_64, AVX512): Recommended AWS instance type: [i4i.4xlarge](#), with at least 16 cores.
- [AMD Zen 3](#) (x86_64, AVX2): Recommended AWS instance types: M6a and C6a.
- [Apple M1](#)
- [AWS Graviton 2](#) (ARM64, NEON): Recommended AWS instance types: M6g, C6g, R6g, and T4g.
- [AWS Graviton 3](#) (ARM64, NEON): Recommended AWS instance types: M7g, C7g, and R7g.

For further questions about hardware recommendations, feel free to ask.

4. Data Collection and Plotting Scripts

- [Plotter](#): Collects data and generates all figures in the paper.
 - Uses data from the repository and additional data produced by running the script.
 - **Note**: The script for *Figure 6* is missing and needs to be completed.
- [Table Generator](#): Generates Tables 4 and 7 from the paper, stores the generated tables in Markdown format in [result dir](#), and prints the tables to the terminal.

5. Tables from Experimental Results

- The master script generates experimental result tables in Markdown format.

6. Documentation

- Detailed instructions for compilation, deployment, execution, and script usage are provided in * [BENCHMARKING](#) * section.
- Guidelines explain how to replicate experiments and benchmarks from the * [publication](#) *.

7. Master Script

- [Master script](#): Automates experiments, data collection, and workflow management.
 - **Outputs**:
 - A log file (master_script.log) with step completion reports (green) and tables (brown).
 - **MacBook Note**: Requires sudo access to enable the cycle counter.
 - **Font Warnings**: May require additional font installation, which does not impact functionality.

8. Deployment and Execution Estimates

- **Deployment time and effort**: X hours
- **Execution time**: Approximately 1 hour for the full workflow (for reviewer preparation).

*Based on *: [SIGMOD ARI Package Requirements and Guidelines](#)

Benchmarking

This guide explains how to:

- Replicate the experiments presented in our [publication](#)

How to Replicate the Experiments

1. Download Data

You can download the datasets used in our publication [here](#) (complete_binaries.zip). These files are in binary format (64-bit doubles) and were used throughout the benchmarks in our paper.

2. Set Environment Variables

Set the environment variable `ALP_DATASET_DIR_PATH` with the path to the directory containing the binary datasets that you downloaded in step 1.

Additionally, inside `data/datasets_transformer.ipynb`, you can find a [Jupyter Notebook script](#) with guidelines to download datasets from their original source and code to convert them to binary format (64-bit doubles). Some datasets may require heavy pre-processing.

3. Set the Correct Toolchain File

There are two toolchain files in the [toolchain directory](#). Adjust them based on the `clang` and `clang++` versions you are using.

4. Run the Benchmark Script

Run the [master_script.sh](#) on the following architectures:

- **Intel Ice Lake (x86_64, AVX512)**: M6i and C6i instances
- **AMD Zen3 (x86_64, AVX2)**: M6a and C6a instances
- **AWS Graviton2 (ARM64, NEON)**: M6g, C6g, R6g, and T4g instances
- **AWS Graviton3 (ARM64, NEON)**: M7g, C7g, and R7g instances
- **Apple M1**: Note that on M1, you should run the following script with `sudo` permissions.

```
./publication/master_script/master_script.sh
```

5. Details:

- [Compression Ratios Experiment](#)
 - [ALP](#)
 - [Chimp, Chimp128, Gorillas, Patas, Zstd Compression Ratios](#)
- [Speed Tests](#)
 - [ALP Encoding Speed Test](#)
 - [ALP Decoding Speed Test](#)
 - [ALP RD Encoding Speed Test](#)
 - [ALP RD Decoding Speed Test](#)
 - [Chimp, Chimp128, Gorillas, Patas, Zstd Speed Test](#)
 - [PseudoDecimals Speed Test](#)
 - [ELF Speed Test](#)
- [FCBench](#)

Compression Ratios Experiment

ALP

```
./publication/source_code/bench_compression_ratio/publication_bench_alp_compression_ratio
```

This target will compress an entire binary file and write the resulting (estimated) compression ratio results (in bits/value) from the datasets in `double_columns.hpp` to the [publication directory](#). One CSV file will be created for the datasets which use the ALP scheme and another one for those which use the ALP_RD scheme. Note that this is a dry compression (compressed data is not stored).

Chimp, Chimp128, Gorillas, Patas, Zstd Compression Ratios

The following target

`https://github.com/cwida/ALP/tree/main/publication/source_code/bench_compression_ratio/bench_{algorithm}_compression_ratio`, in which `algorithm` can be: `chimp|chimp128|gorillas|patas|zstd` will create a csv file for each encoding and for each dataset on the `publication` directory. Note that this is a dry compression (compressed data is not stored). For PDE and ELF, we used their own code for compression ratios.

Speed Tests

All of these tests read the CSV samples files locations from the dataset array. Therefore, to test with your own data, add your dataset to this array. Note that these experiments are performed on 1024 values. Why? Check Section 4 of the [publication](#).

ALP Encoding Speed Test

Encoding is comprised of the `encode`, `analyze_ffor`, and `ffor` primitives. Benchmarked by running:

`https://github.com/cwida/ALP/tree/main/publication/source_code/bench_speed/bench_alp_encode`. Results are located on `publication/results/`.

ALP Decoding Speed Test

Fused decoding is comprised of the `falp` and the `patch_exceptions` primitives. Unfused decoding is comprised of the `unffor`, `decode` and `patch_exceptions` primitives. Benchmark both fused and unfused at the same time on different implementations and Architectures/ISAs by running the commands below. Results are located on `publication/results/`.

Implementation	Command
Scalar	<code>https://github.com/cwida/ALP/tree/main/publication/source_code/generated/fallback/scalar_nav_uf1/fallback_scalar_nav_1024_uf1</code>
SIMD	<code>https://github.com/cwida/ALP/tree/main/publication/source_code/generated/{Arch}/{Arch}_{extension}_intrinsic_uf1/{Arch}_{exter</code>
Auto-Vectorized	<code>https://github.com/cwida/ALP/tree/main/publication/source_code/generated/fallback/scalar_aav_uf1/fallback_scalar_aav_1024_uf1</code>

While the *correctness* can be tested by running:

Implementation	Command
Scalar	<code>https://github.com/cwida/ALP/tree/main/publication/source_code/generated/fallback/scalar_nav_uf1/fallback_scalar_nav_1024_uf1</code>
SIMD	<code>https://github.com/cwida/ALP/tree/main/publication/source_code/generated/{Arch}/{Arch}_{extension}_intrinsic_uf1/{Arch}_{exter</code>
Auto-Vectorized	<code>https://github.com/cwida/ALP/tree/main/publication/source_code/generated/fallback/scalar_aav_uf1/fallback_scalar_aav_1024_uf1</code>

The source file of the `falp` primitive (FUSED ALP+FOR+Bitpack generated by [FastLanes](#)) for each different implementation are at:

Implementation	Source File
Scalar	<code>https://github.com/cwida/ALP/tree/main/publication/source_code/generated/fallback/scalar_nav_uf1/fallback_scalar_nav_1024_uf1</code>
SIMD	<code>https://github.com/cwida/ALP/tree/main/publication/source_code/generated/{Arch}/{Arch}_{extension}_intrinsic_uf1/{Arch}_{exter</code>
Auto-Vectorized	<code>https://github.com/cwida/ALP/tree/main/publication/source_code/generated/fallback/scalar_aav_uf1/fallback_scalar_aav_1024_uf1</code>

Architectures and ISAs:

Architecture {Arch}	ISA {extension}
arm64v8	neon
arm64v8	sve
wasm	simd128
x86_64	sse
x86_64	avx2
x86_64	avx512bw

ALP RD Encoding Speed Test

Encoding is comprised of `rd_encode` and two calls to `ffor` (for both the left and right parts). Benchmarked by running: `https://github.com/cwida/ALP/tree/main/publication/source_code/bench_speed/bench_alp_cutter_encode`. Results are located on `publication/results/`.

ALP RD Decoding Speed Test

Decoding is comprised of two calls to `unffor` (for both the left and right parts) and the `rd_decode` primitives. Benchmarked by running: `https://github.com/cwida/ALP/tree/main/publication/source_code/bench_speed/bench_alp_cutter_decode`. Results are located on `publication/results/`.

Chimp, Chimp128, Gorillas, Patas, Zstd Speed Test

Benchmarked both decoding and encoding by running `https://github.com/cwida/ALP/tree/main/publication/source_code/bench_speed/bench_{algorithm}`, in which `algorithm` can be: `chimp|chimp128|gorillas|patas|zstd`. Results are located on `publication/results/i4i`.

PseudoDecimals Speed Test

We benchmarked PseudoDecimals within BtrBlocks. Results are located on `publication/results/i4i`.

ELF Speed Test

We benchmarked Elf using their Java implementation.